

Mediadesign Hochschule

Game Design

Informatik II

Informatik II

Level Design Tool

Level Design Tool

Supervisor: Matthias Kuhlmann

Andres Pedroza

Street: Ahnfeldstraße 43

Zip Code: 40239, Düsseldorf

E-mail: anpedroza@stud.mediadesign.de

Matriculation number: 205121353

Telephone number: 017660869856

Degree program: Game Design

Düsseldorf, 31.08.2026

Affidavit

I hereby declare on oath that I have written this thesis independently and exclusively
using the works listed in the list of sources and references.

29.08.2026, Düsseldorf

A handwritten signature in black ink, reading "Andes Pedraza". The signature is written in a cursive style with a large initial 'A'.

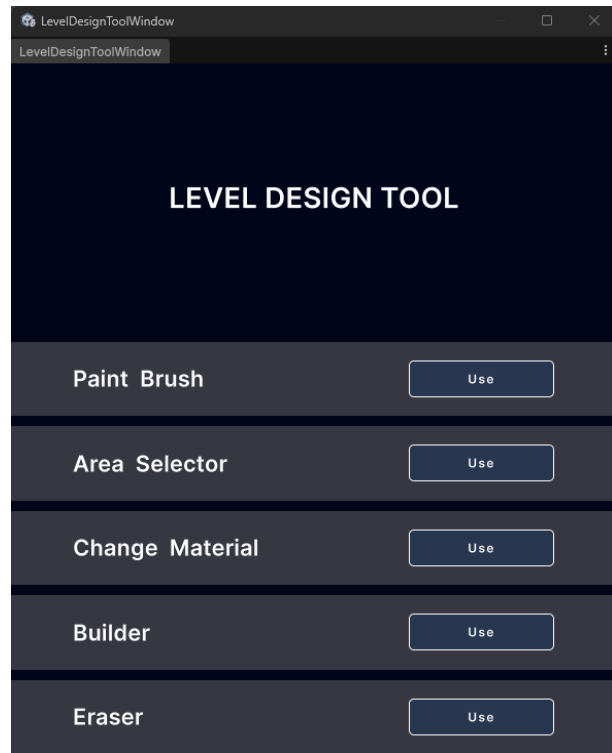
Date, Place

Signature

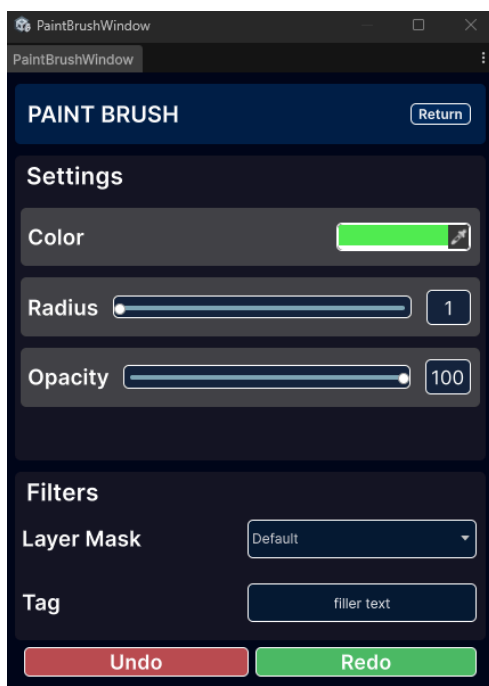
1. System Description

Level design often requires repetitive operations such as modifying materials, placing objects, moving groups of objects, or deleting objects. Performing these operations manually in the Unity Editor can become time-consuming, especially when working with large environments. The goal of this project is to create an editor-based level design tool that allows these operations to be performed directly inside Unity without entering Play Mode. This tool includes the following features:

- Paint Brush
- Area Selector
- Change Material
- Builder
- Eraser



1.1 Paint Brush



This tool allows the user to change the color of any object inside the radius of the brush simply by clicking on it. This brush has the following settings:

- Color
- Radius
- Opacity

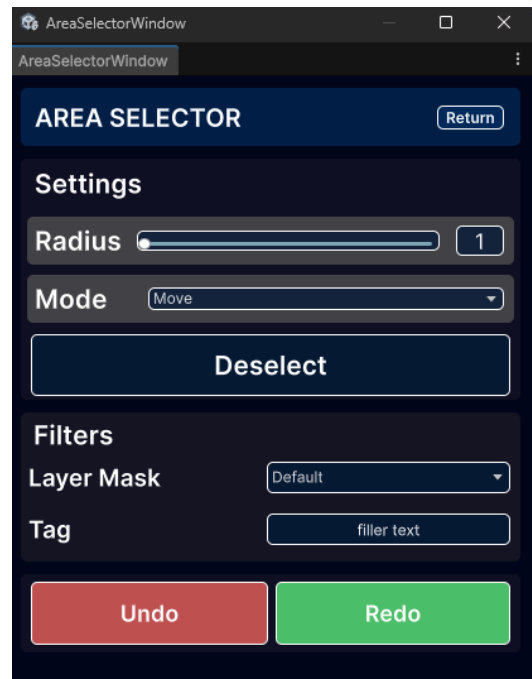
The radius is a slider ranging from 1 to 10, and the opacity ranges from 1 to 100.

1.2 Area Selector

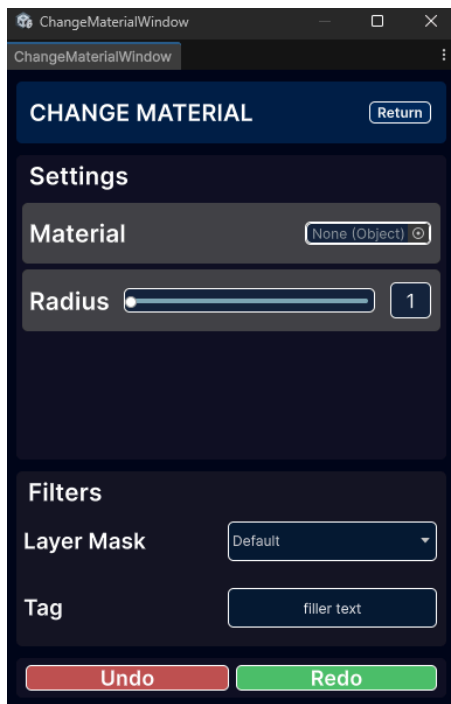
This tool allows the user to select all objects within the radius so that they can later be moved, rotated, or scaled simultaneously. This tool has the following settings:

- Radius
- Mode
- Deselect

It also has a “Deselect” button that allows the user to deselect all objects and the radius is a slider ranging from 1 to 10.



1.3 Change Material



This tool allows the user to select a material by clicking on an object or adding it manually, and then apply that material to any other object within the radius range. This tool has the following settings:

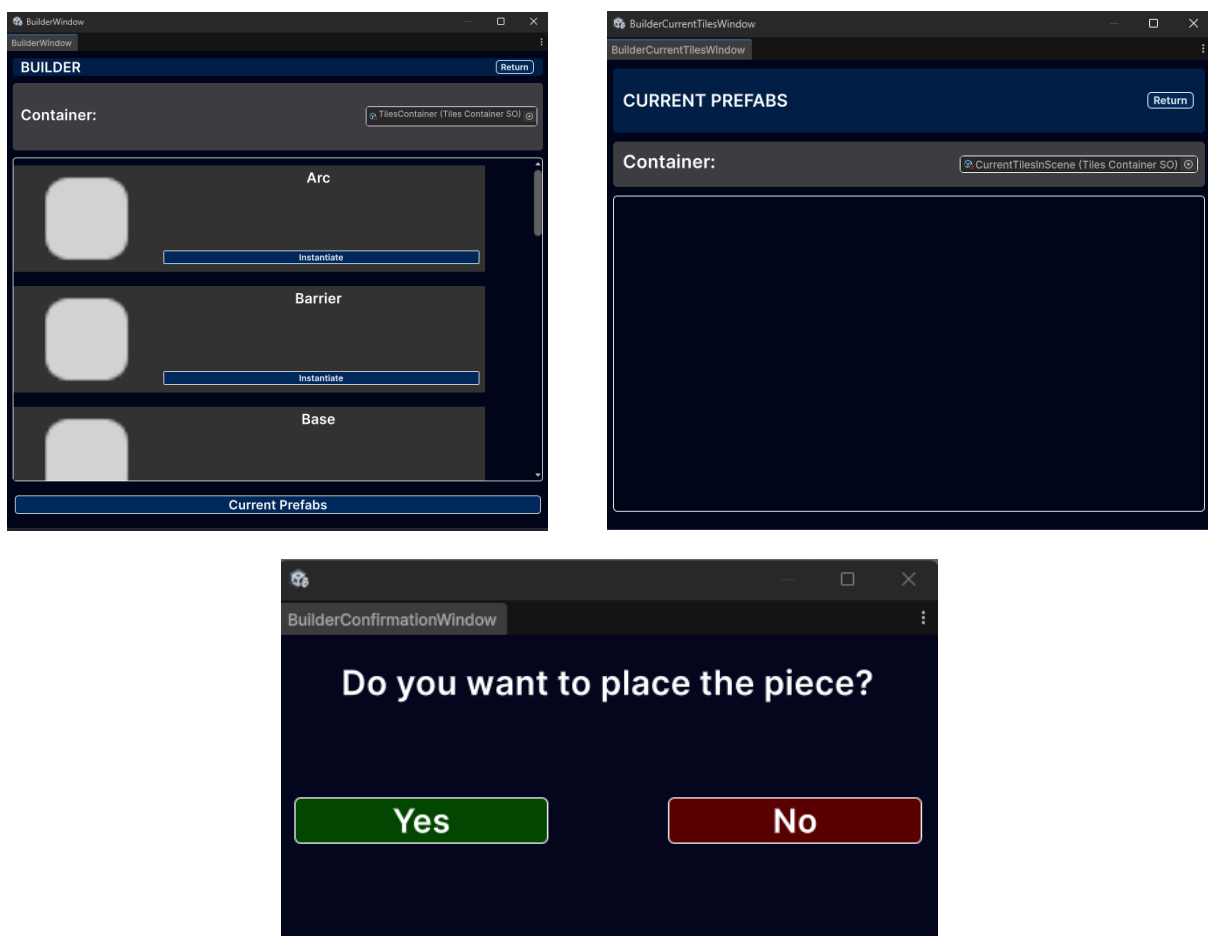
- Material
- Radius

Material indicates which material the user currently has selected and the radius is a slider ranging from 1 to 10.

1.4 Builder

This tool allows users to instantiate the prefabs they have created more quickly and easily. Users should add the prefabs they want to the SO Tiles Container; then, all the objects will appear in a list from which users can choose which ones to instantiate. There is also a tab where users can view the prefabs currently in the scene, as well as a confirmation window.

When the user selects an object, they will be able to see, using a gizmo, a projection of the object's final position. Finally, the user must confirm that this is the desired position.



1.5 Eraser

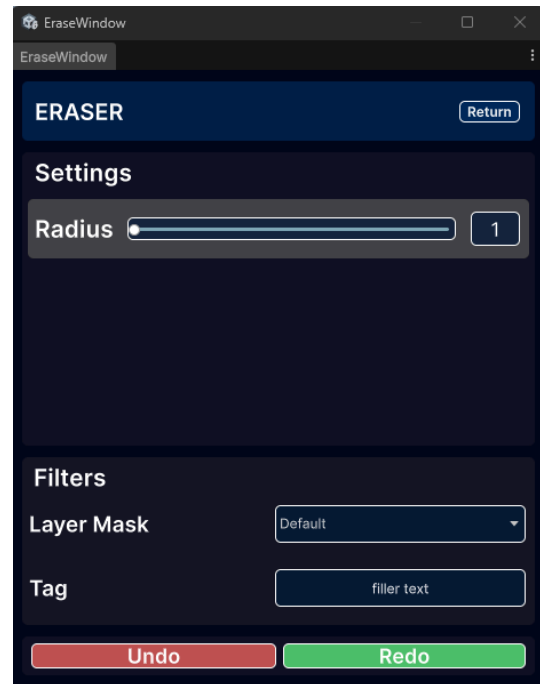
This tool allows the user to delete any object within the specified radius. This tool has the following settings:

- Radius

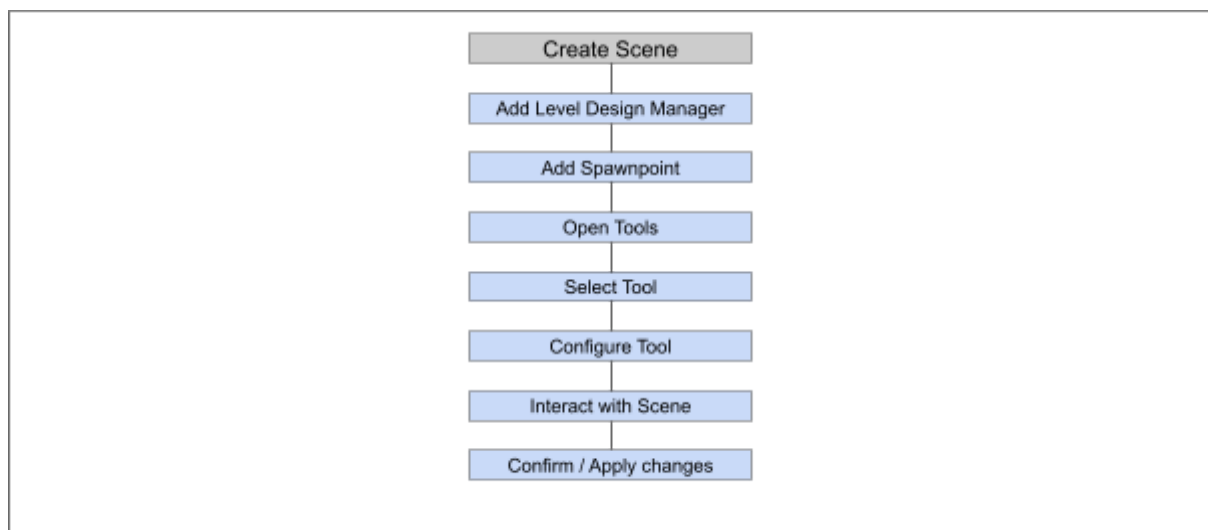
The radius is a slider ranging from 1 to 10.

All tools have a filter that allows the user to specify certain characteristics that the object must have in order for the tool to detect it. These characteristics are:

- Layer Mask
- Tag
- Undo
- Redo



2. Workflow



To use the tool, the user must create a new scene and ensure that both the Level Design Manager and Spawnpoint prefabs are in the hierarchy, these will be located in the “prefabs” folder. After that, the user can access the Level Design Tool main window at the top under “Tools.” Once inside the window, the user can navigate among the different tools and begin using them. After that, the user can select the desired tool, they can adjust the settings to their liking and this will automatically create a GameObject within the scene with the tool's

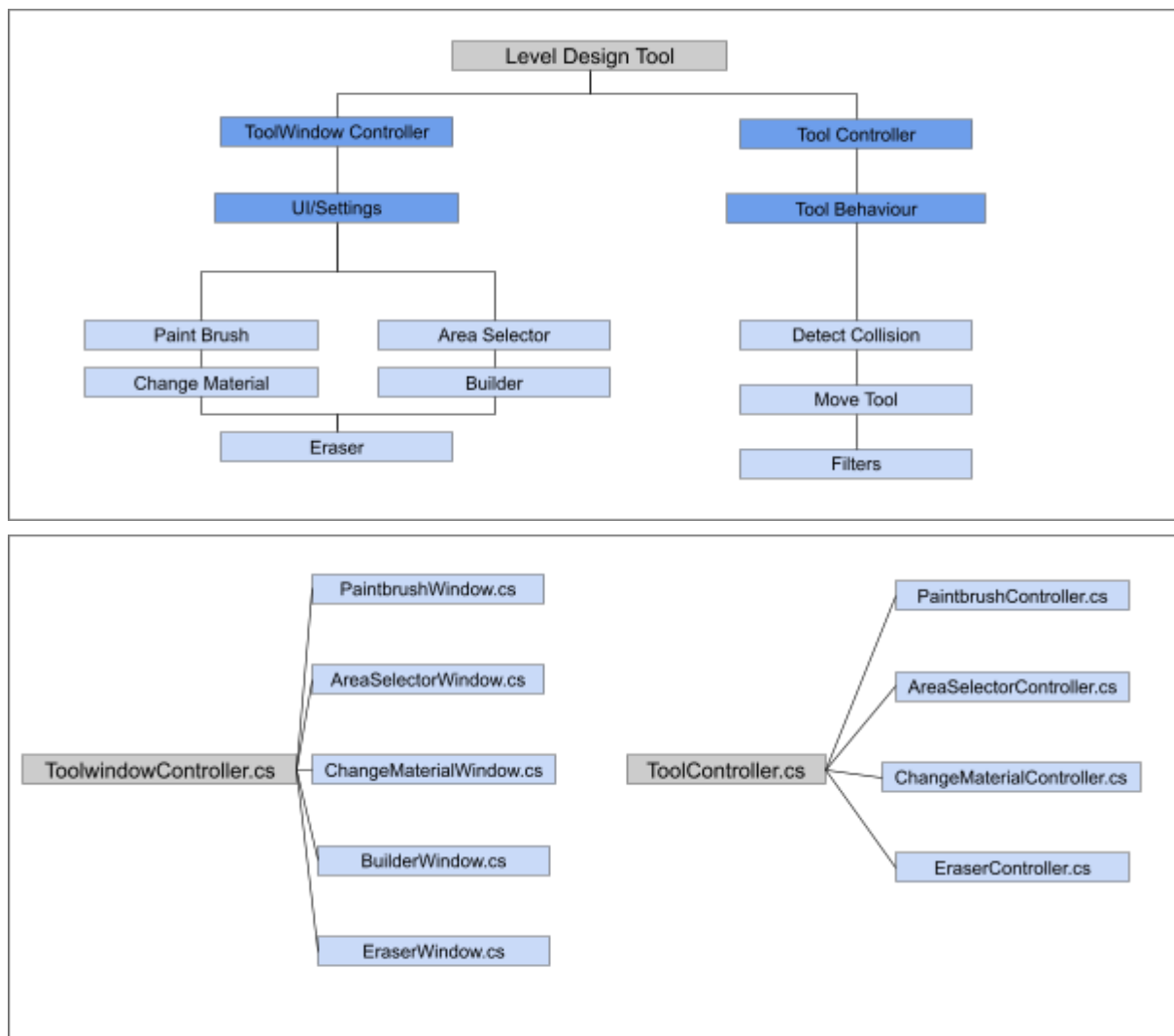
name. This object is important because it will visually indicate where the tool is located and will allow you to detect objects. Then the user can begin interacting with the scene.

3. Core concepts

3.1 Editor

To allow the user to use all these tools directly from the editor, I had to create an “Editor” folder and place all my scripts that require the Editor and EditorWindow classes there. This way, Unity can detect that those scripts belong solely to the editor and not to play mode.

3.1.1 Architecture



For the architecture of the project, I chose inheritance because all tools share common functionality, such as detecting objects, checking filters and controlling the tool interface. By moving this functionality into two base classes, individual tools only need to implement their specific behavior.

- ToolWindowController.cs
- ToolController.cs

On the one hand, the *ToolWindowController*, as its name suggests, controls the tool interface, which means its appearance and the ability to change various settings and filters. On the other hand, the *ToolController* controls the tool itself, for example it detects when the user selects an object.

Inside *ToolWindowController*, there are methods such as:

- protected void Heading(VisualElement ui, EditorWindow closeWindow);
- private void Return(EditorWindow closeWindow);
- protected void ChangeWindow(UnityAction newWindow, EditorWindow currentWindow);
- protected int LoadDataInt(string name);

I designed these methods so that they could be reused in the child scripts for each tool, thereby avoiding the need to repeat the same code in each one. This approach also helps keep the work more organized, clean, and scalable.

Inside *ToolController*, there are methods such as:

- protected void MoveTool(Transform tool);
- protected RaycastHit[] DetectCollision(Vector3 origin, float radius);
- protected bool CheckLayerMask(GameObject gameObject, LayerMask requireLayer, string tag);

Each tool had two scripts: one that controlled the tool's window and another that performed the actions. Two of the most important methods in the *EditorWindow* class are *OnGUI()* and *CreateGUI()*, these are equivalent to Update and Start in the MonoBehaviour class.

Within the Window classes for each of my tools, the process is very similar. The first thing I do in *CreateGUI()* is create a clone of the rootVisualElement so I can display my UI. Then I assign values to each of the elements, for example, the radius, tag, layer, etc. I also instantiate a GameObject that contains the Tool scripts to display the gizmo within the editor.

Finally, within *OnGUI()*, I constantly change the values of the different UI elements and save them using *EditorPrefs*.

In addition, all tools except the builder have an Undo and Redo feature. This will allow the user to revert to previous changes.

3.1.2 Paint Brush

My *PaintBrushController* class is designed solely to paint game objects, which is why it contains a method called *PaintTile()*. This method allows me to detect when the user clicks, then check whether the mouse has collided with any object using the *DetectCollision()* method found within the *ToolController*. Once the layer and tag have met the requirements using *CheckLayerMask()*, the tile is painted by changing the color of its material.

3.1.3 Area Selector

My *AreaSelectorToolController* class is designed to select and deselect different objects within the scene. To do this, I have the *SelectObjects()* and *DeselectObjects()* methods. *SelectObjects()* checks all the objects that were detected when the user clicked and then adds them to the selectedGameObject list. *DeselectObjects()*, on the other hand, checks all the objects in that list and removes them.

Once the user has selected the desired objects, one of these three functions is triggered: *MoveObject()*, *RotateObject()*, or *ScaleObject()*. To do this, I used an enum containing the three options the user can choose from the window and created a dictionary that contains both the enum and the function corresponding to that enum. This allows me to call *_ModeDictionary[currentMode].Invoke()* within the *Update()* method. All three methods call *EditorGUI.BeginChangeCheck()* and *EditorGUI.EndChangeCheck()*, while using the Handles class, which provides methods such as *PositionHandle()*, *RotationHandle()*, and *ScaleHandle()*.

3.1.4 Change Material

My *ChangeMaterialToolController* class is designed solely to change an object's material, for that reason, it contains a method called *ChangeTileMaterial()*. This method works very similarly to the paintbrush. First, it detects whether the user clicked, then, it checks for a

collision, and finally, it changes the object's material by accessing it through the object's renderer.

3.1.5 Builder

My Builder tool consists of 3 classes. First, there's *BuilderWindow*, where the UI is generated using the same method of cloning the *rootVisualElement* as in the other tools. In addition, I have a method called *PopulateScroll()*, which allows me to populate each tile with information from the various prefabs within the game. Finally, I have a method called *InstantiateTile()*, when you click the "Instantiate" button, it creates a new *GameObject* and opens the confirmation window.

To implement this confirmation window, I created the *BuilderConfirmationWindow* class, which has only two functions: to confirm or cancel the operation. The *Confirm()* method adds the tile to the scene and to the list of current *GameObjects* within the manager, then returns the user to the builder window. The *Cancel()* method destroys the object and also returns to the builder window.

Finally, the *BuilderCurrentTilesWindow* class contains only one method, *PopulateScroll()*, which is used to add tiles containing information about the various prefabs in the scene. This method works very similarly to the one in *BuilderWindow*.

3.1.6 Eraser

My *EraserToolController* class is designed solely to remove an object from the scene and delete it from the list of current objects managed by the object manager, for this reason, it contains a method called *EraseObjects()*. This method works very similarly to the paintbrush and change material methods. First, it detects whether the user clicked, then, it checks for a collision, and finally, it removes the object using *DestroyImmediate()* and *Remove()*.

3.1.7 UI-Toolkit

To create a UI for the windows, I had two options. The first was to create it using only code through the *EditorGUI* and *EditorGUILayout* classes, the second was to use the Unity UI Toolkit. The problem with the first option is that I couldn't see a visual preview of what the design would look like, and it also had more design restrictions and was more tedious to create. In contrast, the Unity UI Toolkit allowed me to create UI using *uxml* and *uss*, which I

could then make functional through scripts. I decided to use this system because I have knowledge of HTML, CSS, and JavaScript, so it was quite easy for me to create the interfaces with it.

The method I used to add functionality to the UI was through `rootVisualElement`, where I had to search for the button or slider using a string.

3.1.8 Save Data

EditorPrefs is used so that the user's tool configuration persists between different uses of the tool. This prevents the user from having to configure values such as radius or layer masks every time the window is reopened. *EditorPrefs* class can be found in the Unity Editor library, which contains methods such as:

- `SetString(name, value);`
- `GetString(name);`
- `SetInt(name,index);`
- `GetInt(name);`

So all I had to do was call these methods when the user changed a value and then load the information when that tool's window was reopened.

3.2 Gameplay

3.2.1 Tiles

To display a preview of what the final position of the pieces would look like once they were placed, I used a script called *TilesController*, which contained only one gizmo. This gizmo rendered a green wirecube mesh showing the piece's position, it also allowed me to render a white mesh of the piece once it was selected.

3.2.2 Manager

The *Manager* is located within an object in the scene called Level Design Manager. This manager contains references to all the *VisualTreeAssets* for the various UIs, as well as a *SO* container for the objects currently in the scene and the spawning of the pieces.

3.2.3 Tools

For the tools within the scene, I create a script called ToolGizmo, which contains only a Gizmo made up of a white wireframe sphere that represents the tool's radius.

3.3 Data-ScriptableObjects

ScriptableObjects were used to separate prefab data from the tool's runtime logic. This allows the Builder to access and display prefab information without storing this data directly inside the editor window. One is a *TilesContainerSO*, which contains a list of GameObjects and allows me to store the various prefabs. The other is a *TilesTemplateSO*, which serves as a template for the data that the various prefabs must contain:

- TileName
- TilePrefab
- TileImage

I use this data when creating the builder window to display information about each of the user's prefabs within that container.

If the user wants to add new prefabs to the *TilesContainerSO*, they must keep in mind that these prefabs must contain the *TilesController* script.

3.4 Libraries & Packages

- UnityEngine
- UnityEditor
- UnityEditor.UIElements
- UnityEditorInternal
- UnityEngine.UIElements
- UnityEngine.Events
- UnityEngine.InputSystem
- System.Collections.Generic
- Grid-Master

4. Difficulties:

4.1 Avoiding duplicated code

One of the first problems I ran into was that much of the code could be reused, since several features were going to be used by all the tools, for example, object detection and the filtering system. To solve this problem, I created the base class *ToolController*, which allowed me to reuse those methods for each of the tool classes.

4.2 Persistent tool settings

Another problem I encountered was that whenever I closed the current window and then reopened it, all the changes I had made were lost. This made using it very tedious, since I had to constantly change the settings. To solve this problem, I used the *EditorPrefs* class, which allowed me to save the information as a string or an integer so I could load it again later.

Moreover, another problem was that once the user made a change, it could no longer be undone. Because of that, I used the *Undo* class, which contains methods such as *Undo.RecordObjects()* and *Undo.PerformUndo()*, allowing me to save an object's information before it was changed and thus undo the changes made by the user.

4.3 Prefab management

Finally, the builder window needed to update dynamically based on the information for each of the prefabs, as well as the available prefabs. To solve this problem, I used an SO as a data container, which allowed me to store that information and then display it in the window without having to enter it manually.

5. Upcoming Improvements

5.1 Current Prefabs

Currently, the "Current Prefabs" window only detects changes if the user creates objects using the builder function. A future improvement will be to enable "Current Prefabs" to detect when the user creates objects from the hierarchy.

6. Github-Link:

<https://github.com/AndresPedroza-Games/Level-Design-Tool>